COMP3850 Computing Industry Project

Table of Contents

Foreword *2*

Project Phases **2**

Team Evaluation **3**

Sponsor Contact **3**

Changes From Previous Versions **3**

Deliverable 1 Details *4*

Deliverable 1 rubric **4**

Deliverable 2 Details *5*

Resubmitting Documents & Document Formatting **5**

Feasibility Executive Summary **5**

Project Plan **5**

Project Progress **7**

Marking scheme for ALL project streams **7**

Requirements Document/ Scoping Document **8**

Common section for all groups 8

Details for CYBERSECURITY projects 9

Details for GAME projects 14

Details for SOFTWARE projects 16

Marking scheme for scoping / requirements sections **18**

Deliverables 3 & 4 ..... *19*

ASSESSMENT 2 - Team Project Rubric **19**

All Teams: Deliverable 3 & 4 submission instructions **19**

D3 & D4 Common sections for all groups **21**

Updated Project Plan 21

Updated Requirements / Scoping Document 22

Project Documentation (Summary – See Individual Sections) 22

Prototype / MVP Document 23

Team Retrospective [D4 ONLY] 24

D3 & D4 Project Documentation - DATA SCIENCE **25**

D3 & D4 Project Documentation - CYBERSECURITY **28**

D3 & D4 Project Documentation - GAMES **31**

Game Design Document – MDA Framework 31

Game Design Artefacts 32

Game Scaffolding a.k.a. Software/System Design Document 32

Game Testing deliverables [Test Specification] 33

D3 & D4 Project Documentation - SOFTWARE **36**

Analysis Documentation 36

Design Documentation 37
Testing Documentation [Test Specification] 39
Deliverable 5 Details 41
Deliverable 6 Details 43
Deliverable 7 Handover 44
SOFTWARE & SUPPORTING MATERIALS 44

Foreword

The purpose of this document is to provide details on each of the deliverables for this unit. As this unit has low contact hours, it is expected that teams manage their time effectively, are aware of due dates, and read this document carefully to understand what is required for each task. More resources such as templates are available on iLearn, and you can always reach out to unit staff if you are unsure about anything.

All team members should be familiar with and refer to this Project Deliverables Definition Document before submitting major assessment items. The team Quality Manager should ensure the team follow the instructions correctly.

As part of COMP3850 you will be formed into groups of 4-6 students. You will work together to understand what your client requires and go through a number of phases from assessing the feasibility, requirements, design, implementation and testing of a prototype or other artefact with associated documentation, as appropriate for your project. You can expect requirements, and client needs to evolve over the period of the project.

Every project is different. These documents have been prescribed in a way that they will both allow for evaluation of you meeting the learning outcomes of the unit, and also be useful in your own work on the project. You may find your client is not interested in every part of these documents. That's fine. Responding to these criteria is a useful way for your team to work on your project. Please see these documents not as "something extra", but as a core part of your project work.

Additionally, there may be times you wish to combine elements from different streams, or supplement them for others. It's important to speak to unit staff about this ahead of time, but generally we are amicable to this as it shows an understanding of the project's needs.

Project Phases

The unit is divided into 3 project phases:

1. **Team Project Induction** – 25% (comprised of induction and team training; feasibility report and initial team manual (D1), project plan and scoping OR requirements document (D2))
2. **Team Project** – 50% (Updated project plan and scoping or requirements document, project documents and MVP (D3 & D4), final presentation/demonstration (D5))
3. **Individual Retrospective** – 25% (Individual written reflection (exam)(D6) and MVP handover and sponsor review (D7)).

Team Evaluation

For both D2 and D4, every student must fill out a Team Evaluation form, stating the percentage, performance and any other notes of each team member for that particular period of time. This is the predominant data point used for individualising grades for these assessment tasks, and so it is important all students take this seriously. Failure to complete the Team Evaluation is vetoing your opportunity to have your say on your performance.

The project will follow an agile life cycle and is made up of a number of deliverables that will be iteratively updated and expanded.

Sponsor Contact

In order to deliver valuable outputs within the semester, you are expected to follow agile processes including regular and frequent communication with your sponsor to obtain their feedback on your ideas and work. Your sponsor is not responsible for editing or assessing your documents. Make sure what is presented to them is professional and businesslike.

Deliverable 1: Discuss the Feasibility Report with your sponsor to ensure any misunderstandings are clarified, particularly concerning **their** problem, opportunity, mandates and assumptions. **After** receiving your Deliverable 1 feedback on iLearn and also updating based on your improved understanding, discuss with the sponsor what parts of the Feasibility Report they are interested in and provide them with either the full report or as a minimum a 1 to 2-page Executive Summary stating the sponsor's problem and opportunity, mandates, current situation, proposed solution and assumptions. What you provide to your sponsor will be submitted as part of Deliverable 2.

Deliverable 2: Before Deliverable 2's due date, you must obtain sponsor feedback to the Requirements/Scoping document. Also include the feedback and what changes were made. This feedback and response should be included in the submission.

Deliverables 3 and 4: Before D3 and D4 due dates, you should have demonstrated your prototype/minimum viable product (MVP) to your sponsor. In the Prototype section, your submission should include when/how the demonstration was done, the feedback you received and your response to it. You will lose marks for your prototype/MVP if you do not include information about the prototype in your D3 and D4 documentation and regarding the meeting with your client, the feedback you received from them and how you will respond.

Changes From Previous Versions

This document has been changed to remove details that are communicated via iLearn or the unit guide, such as due dates and unit schedule.

Deliverable 1 Details

Deliverables Certificate
Feasibility Report approx. 8-15 pages
Team Manual approx. 3-6 pages

Please liaise with your industry sponsor about their needs, problem or opportunity description and the solution sought. There may have been supporting documentation you received. The team should do further research to understand better the organisation, domain, problem, task, solution options, etc. Based on the information given and your research, discuss the problem, the activity/task, the domain and issues with your team so that you get a bigger picture of the context the problem fits in. Particularly consider the IT needs related to that activity and develop an initial proposed IT solution to one or more of the problems. You will need to describe the problem, alternative solutions (even if a specific solution has already been requested) and justify the solution you recommend.

Use the templates on iLearn for your Feasibility Report and Team Manual. Please ensure your Deliverables Certificate states which type of project you are doing: software, game, data science or cybersecurity. Assumptions – it is important to include assumptions. If those assumptions are wrong, then your plans and project will be compromised. Assumptions could include access to certain resources, scope of the system, support that will be provided, platform that will be used, etc. It is good to elaborate these so the sponsor can see them and correct any false assumptions.

After receiving your Deliverable 1 feedback on iLearn and also updating based on your improved understanding, discuss with the sponsor what parts of the Feasibility Report they are interested in. Provide them with either the full report or as a minimum a 1 to 2-page Executive Summary stating the sponsor's problem and opportunity, mandates, current situation, proposed solution and assumptions.

Deliverable 1 rubric

Problem Identification & Project Classification
Opportunities
Mandates
Current Situation
Tangible & Intangibile Benefits
Alternative Solutions
Recommended Solution
References/Abbreviations/Assumptions
TOTAL
Team Manual
Team Organisation and Structure
Team Values & ACS Code of Professional Ethics
Project Management Approach and Tools
Communication Plan and Meeting Schedule
Change Management and Conflict Resolution

For your Team Manual, discuss team structure and roles. Review together the 2023 ACS Code of Professional Ethics (see iLearn). Describe the team's values and how you will adhere to the Code. Clarify and justify the approaches and/or tools that will be/are being used for project management, team communication and to manage change or conflict. Bear in mind the 2023 ACS Code of Professional Ethics as you consider mandates, current situation, benefits, alternative solutions and your recommendation. There will also be a column indicating if a section is Poor, Okay, Good, or Excellent to give you an idea of the quality of the work. The work done on the team manual will be relevant later for incorporation into the project plan.

Note that for each deliverable, teams are responsible for investigating what content is needed, needs updating and should be continuously refining their documents. The feedback from this deliverable is not to be solely used as a checklist of items to fix for the next deliverable, as they should be revising their documents independently and within the context of the state of their project and current goals.

Deliverable 2 Details

Deliverables Certificate
Feasibility Executive Summary provided to sponsor (3.5 marks)
Project Plan (11 marks)
Project Progress (3 Marks)
Software Requirements Specification (SRS) Document **OR** Scoping Document (7.5 marks)

Teams will submit either:
Scoping Document (Data Science and Cybersecurity streams) OR
Requirements Document (aka Software Requirements Specification (SRS) for Software and Games streams)

Resubmitting Documents & Document Formatting

Do not assume if you address sponsor and prior deliverable feedback that you will get full marks. If this was so, then all teams should receive full marks for Deliverables 2 and 4. You can't expect others to tell you how to do something excellent. A good or adequate report/document (Pass) is not the same as an outstanding/exceptional one (HD). As a team, review feedback and also consider how you can improve what you have done. All updated documents require their own revision history table to indicate everything that has changed (include a detailed explanation of the reasons for the changes and mention the username of the person who performed each task). Be professional and consistent in document formatting: level headings, header, page numbers, snapshots/images/figures/tables need captions/labels and to be referred to, consistent standard font size and spacing, use of tables and bullet points to aid readability, but not overdone, avoid splitting tables/figures. The content should be concise and clear, coherent and flow logically, spelling and grammar checked.

Feasibility Executive Summary

You should have shared your full report or Executive Summary with your industry sponsor after reviewing and revising according to the team's improved understanding and Deliverable 1 feedback. For D2, submit what you have provided to your industry sponsor. Unit staff will check that your sponsor has received this document.

Project Plan

The Project Plan should not exceed (20) A4 sides, excluding appendices. See the marking rubric for content to be included and follow these headings and order.

Your *Project Plan* should consist of the following (in the outlined order):

1. **Statement of purpose [for plan] and scope [of project]**
2. **Risk Management**
    - Development risks and their management, including a risk matrix/table that includes the risk, description, probability/likelihood, effect/consequence and risk level. Mitigation strategies must also be included.
3. **Resource Management**
    - People: Describe team values, structure, roles and responsibilities for all team members. Identify all stakeholders and communication patterns and conflict resolution strategies.
    - Hardware, software, other resources (Include separate subheadings for each). This section should include tools to be used, identifying their purpose, version & version control software, source/access, etc
        - Note: If specialised/proprietary software is required (e.g. a Game Engine), some mention of license/seat allocation is expected (which

developers have access to the software); if a server/database is required, who has access to it (and what level of privilege); if physical access is needed, how is this being sorted (are they visiting the site fortnightly etc); if other hardware is required (e.g. VR Devices), how is this being sorted, how are teams sharing/giving hardware out etc

4. **Change Management**
    - Changes to requirements and scope
    - Change to documents, code and data (i.e. version control).
5. **Quality Management**
    - Discuss how quality will be monitored and assured (i.e. A Quality control plan)
6. **Schedule**
    - Using the project management tool of your choice (e.g. Trello, MS Project, ClickUp), include chart/screens that clearly show planned tasks, unit deliverables, resource/task allocations etc. for the entire project.
        - We are after evidence that appropriate planning has taken place
        - Large tasks (i.e. singular tasks that would take more than a week to complete) should be broken down into smaller tasks.
        - Your schedule must be consistent with both unit deadlines and deadlines from the process model chosen
    - Clarify the process model you are using and why - Software and Games teams, See further NOTE below.
    - Description of the purpose of tasks/milestones.
        - This could be a brief, one sentence description per task
        - This could also be a description of a larger milestone, and an outline/list of tasks required for this milestone
    - Some level of analysis regarding task dependencies. Ideally this is simply visualised in your included chart/screens.
7. **Assumptions**
    - Include a list of assumptions around resources, availabilities, tools, techniques, processes, standards, communication, expectations and anything else your project relies on or assumes. This will help you think about them and whether they are reasonable and also help the marker/review team to understand why you have done certain things.
    - Please review the assumptions as a group before submission.
8. **Standards/Templates/Appendices/Forms**
    - Some teams will include standards e.g. coding or document standards, forms, etc as part of their project plan to provide further guidance and detail. These are not specified as each project will be different. They are also optional. But if included they should be referred to in the main document so the reader knows they exist and how they can be used.

NOTE: For software development and games projects, we are using an agile software development life cycle (SDLC) (i.e. process model) because the requirements are often not fully known and will emerge as the project progresses based on the client's feedback. Data science projects should follow the intent of an agile data science process (http://www.datascience-pm.com/what-is-agile-data-science/). Agile approaches (e.g. SCRUM and Kanban) are widely used in industry because agile methods allow all stakeholders to communicate and validate their ideas sooner. Research what agile methodology best suits your project. Additionally, while 'story points' are not required as an inclusion, you will be marked down if your tasks are too 'large'.

Project Progress

Provide a summary (up to one-page) that describes what the team have done so far related to the project, which is additional to the Feasibility Report, Team Manual, Project Plan and Requirements/Scoping Document. Consider the following and use these headings:

- **Research and Training** – what has been investigated? What knowledge and skills gaps have been identified? How will they be filled? What training/preparation has been done.
- **Project Tasks and Activities** – What has been started/completed so far?
- **Testing, Verification and Validation** – what testing, V & V will be needed? What has been done so far? What feedback has been received and how has/will it be acted upon?

These activities, and who did them, should be listed in the Timesheet which is part of the Deliverables Certificate.

Please provide the above documents in the order listed above and, in the rubric, below.

Marking scheme for ALL project streams

Feasibility Report / Executive Summary	**3.50**
Problem Identification /Opportunity/Benefits	1.25
Mandates and Current Situation	1.0
Proposed Solution	1.25
Project Plan	**11**
Statement of Purpose/Scope of Project/Description	1.0
Risk Management	1.25
Resource Management	
People	2
Hardware, Software, Other	1.5
Change Management	
Managing requirements and scope change	1
Version Control	1
Quality Management	1
Project Schedule	
Tasks/Deliverables/Process	1
Timeline	0.5
Resources and Tasks allocated	0.25
Assumptions	0.25
Standards/Templates/Appendices/Forms	0.25
Project Progress	**3**
Research & Training	1
Project Tasks and Activities	1
Testing, V & V	1

***** Note: This is not the entire rubric for this deliverable.*** *The following pages contain information regarding the* ***Requirements / Scoping Document,*** *which is stream specific and includes further specifications and rubrics for this deliverable.*

Requirements Document/ Scoping Document

(6-20 pages)

Common section for all groups

You must provide the requirements/scoping document to your client before the due date to receive feedback from them.

The marker will assume what they receive has been confirmed by the client. You can expect to make changes to your document based on the client's review, so **make sure that it is drafted early;** early enough that your client has an appropriate amount of time to give you feedback, and you can address it.

It is best, though not essential, to send your draft document to the client AND also arrange to meet them to obtain face-to-face comments. **Your client may also like to see your project plan** - ask them. Make sure you clarify and confirm with them any resources you identified in the project plan that you expect from the client.

Include at the END of your Requirements/Scoping Document a page that outlines:

- **Meeting date and time**
- **Feedback received** – If you don't hear back before the due date, clarify that in your document, say when it was provided to the client and clarify that you will provide the feedback and responses in your weekly report once you receive them.
- **Team response/action plan** - You do not have to do the changes requested before submission but you should outline what action you will take.

Additionally, choose which of the following streams to follow for your scoping document/requirements document:

- **Cybersecurity**
- **Data Science** (*may be best choice for AI projects*)
- **Game Development**
- **Software** (*Default if unsure / not specified otherwise*)

Specifics regarding these streams & rubrics can be found in the sections below.

Details for CYBERSECURITY projects

(6-20 pages)

The scoping document outlines the purpose and direction of your project. Its goal is to capture your initial analysis of the problem space. It should clearly communicate key details to your team, unit staff, and project sponsors or stakeholders.

Cybersecurity Scoping Doc.	**Mark**
Document convention/intended audience	0.25
Purpose/Project Scope	0.25
Attacks/Vulnerability Understanding	0.5
Adversaries	0.5
Attacks/Vulnerability Detection	0.5
Monitoring of Assumptions	0.25
Data quality	0.5
Data Preparation	0.5
Data (packages, logs, …) Collection	0.25
Security Feature Extraction	0.5
Attacks/Vulnerability Analysis	0.5
Defence countermeasures	0.25
Architecture, Algorithms, Models,	0.25
Security Testing (e.g., penetration testing)	0.5
Deployment – monitoring, reporting, review, training/support, security polices)	0.5
Client/Partner Feedback – was received by the client reasonably before the due date & feedback from the client was given & included	1
Follows Format Specified/Well-presented	0.5
TOTAL	7.5

The guidance in the sections below will help you understand what to include and how to structure your document.

Note: For rubric sections specifically related to data, please refer to the **Data Understanding** and **Data Preparation** points in the Data Science guidance section.

Part 1: Analysis overview

(intent / motivating factors)

1. **The purpose and scope of activities being considered**
   - What systems are being investigated / profiled / attached / guarded and why?
   - What kind of project are you undertaking?
     - An attacking team? (red team)
     - A defensive team? (blue team)
     - An overall security assessment analysis with recommendations?
     - (Any additional noteworthy inclusions)
2. **The understanding (high level overview) of attacks and vulnerabilities** that are going to be investigated
   - What attack vectors are being considered / reviewed / researched / attempted?
3. **The types of adversaries that will be considered / profiled** as part of any defensive or offensive strategy the team will be undertaking
   - This is akin to grouping adversary profiles into different groupings (e.g. what are the different kinds of bad actors and what would their goals be?). This can help to consider how to guard against certain types of attackers and preventing their intended outcomes.
   - What resources would each of these different adversary groups have at their disposal to achieve their outcomes?
4. **The detection methods that are being considered / are available for investigation**.
   - You need to understand the important role of threat detection in any security system/solution. Even if your project is a simple application/plugin, if you don't pay attention to detection, then if an incident happens, you will have no clue how it happened. And even worse, your system/solution has no embedded mechanism to respond to threats before they become incidents.
     - If this has not been part of your considerations in the final product, it is highly recommended to include Threat Detection function. For instance, in providing the proposed end-to-end encryption and storage solution, is there any chance that someone would be able to intercept or tamper the data transmission? If so, how would your solution respond to that? What do you do to prevent these kind of threats from happening again?
5. **Monitoring of assumptions**
   - Describe of how the identified assumptions will be monitored in your project to ensure they have not been invalidated.
6. **Document conventions & Intended Audience**
   - Document convention should explicitly define structure, language usage, and acronyms.
   - The intended audience is important for scoping context and tailoring explanations – this is useful for anyone who doesn't know what to expect in this document.

Part 2: Analysis activity details / specifics

(mechanisms for investigation being considered)

1. **Security feature extraction**
   - This should expand on the points in (Part 1.1). and (Part 1.2).
   - What security features / weaknesses are initially being considered?

Note: Part 2.1 (above) focuses on your solution, i.e. what security features you will provide in the final product.

2. **Attack / vulnerability analysis**
   - This should expand on the points outlined in (Part 1.2).
   - In Part 1, you outline attacks / vulnerabilities being investigated
   - Here in Part 2.2, is a chance to answer questions like
     - "What does this look like in the systems we are focusing on"?
     - Are there any indicators that might be a sign that this kind of attack / intrusion is currently occurring?
     - Are there any impacts / results which can show that such an attack / intrusion has occurred?

Note: Part 2.2 (above) answers why you want to provide these security features by outlining the attacks/vulnerabilities that you have investigated in the current system (or in other solutions that you have investigated but didn't choose).

3. **Defence countermeasures**
   - After identifying those points in Part 2.2, now it is time to consider what defence countermeasures that could be used / developed as part of any strategy.
     - If you are a red team project, then consider how these countermeasures might impact your approach.
     - If you are doing an overall security assessment / providing recommendations, then consider how any current defence mechanisms may impact your investigations / obscure other issues / limit the investigation capabilities.
   - This is also a chance to have more details on the points outlined in Part 1.4.

4. **Architecture, Algorithms, and models**
   - This is an expansion / elaboration on (Part 1.4). In that section, you've outlined the potential detection methods being considered, here is where you elaborate on these in more detail.

5. **Security testing**
   - What are the different kinds of tests / assessments being considered? (this may change over time as more is discovered about the systems under focus).

6. **Deployment / monitoring / reporting**
   - Intended approach for deploying / running any tests or analysis scripts
   - How the monitoring of progress and results will be undertaken
   - What reporting / results / recommendations are being provided to the client.

Note: Cybersecurity majors should have gained experience with a range of methods, tools and documentation in your previous units. Your client is also likely to have examples and preferred methods, tools and documentation to be used in your project. Please discuss these with your client and describe them in the section on "Deliverables Due" in your project plan. You will need to extend the project plan to a more comprehensive scoping document. Please refer to the marking rubric for content/sections in your scoping document.

Details for DATA SCIENCE projects
(6-20 pages)

Data Science Scoping Doc.	Mark
Document convention/intended audience	0.25
Purpose/Project Scope & Context Diagram/	0.25
Data Understanding – describe the data	0.5
Sources	0.5
Capture	0.5
Storage/Environment	0.25
Data quality	0.5
Data Preparation - cleaning needed	0.5
Integration needed	0.25
Formats needed	0.5
Data Exploration – Modelling Modelling technique selection	0.5
Model design constraints	0.25
Model assessments needed	0.25
Evaluation Requirements & Process	0.5
Deployment – monitoring, reporting, review, training/support)	0.5
Client/Partner Feedback – was received by the client reasonably before the due date & feedback from the client was given & included	1
Follows Format Specified/Well-presented	0.5
TOTAL	7.5

Data science projects will need to extend the project plan into a more comprehensive scoping document by adding the extra sections as described below. You will need to follow a project management method such as CRISP http://www.datascience-pm.com/crisp-dm-2/ - For phase 1 (Business Understanding) you have already been asked to create a feasibility report and a project plan.

In your scoping document, your team is aiming to communicate the intended scope of work. However, it's expected that this will evolve as the project progresses — this is entirely acceptable. The scoping document is a living resource, and should be regularly reviewed and updated in collaboration with sponsors/stakeholders.

In the scoping document, include each of the following sections:

1. **Data Understanding**: collect initial data; describe data; explore data; verify data quality
    - What are the initial data sources being used / provided?
    - What data collection methods are being used?
    - What is the assumed data quality / structure / completeness?
    - What mechanisms will be undertaken to evaluate the above points? Will you have all the data that you need or will there need to be more data collection methods done during the project?

2. **Data Preparation** (generally, the most time-consuming phase): select data; clean data; synthesize data; integrate data; format data
    - Describe the intended data preparation activities for each of the data sources listed above
    - Describe any data processing pipelines that will need to be done. E.g. noise filtering, smoothing, running averages, error corrections, removing outliers, anonymising any datasets, feature engineering procedures…
    - If there needs to be different preparation pipelines for different models, then describe the processes needed for each model.
    - Also, describe why these steps would make the data more usable for the intended purposes.
    - What data storage mechanisms will need to be used during these processes?

3. **Modelling**: select modelling technique; generate test design; build model; assess model
    - Which modelling technique / techniques are being used / compared? Why? What is the intended outcome / question being considered and how does it relate to the chosen model?
    - How will the model effectiveness / validity be checked?
    - What test data needs to be provisioned / used for the above points?

4. **Evaluation**: evaluate results; review process; determine next steps
    - How will the results be reviewed? Are there expectations to be compared to, or is this more of an open exploration / unknown result? If so, how will these results be reviewed / validated?
    - What next steps or questions are being considered depending on the results / outcomes?

5. **Deployment**: plan deployment; plan monitoring and maintenance; produce final report; review project.
    - What is being given to the client? What training / documentation will be provided?

Details for GAME projects

(6-20 pages)

Games projects are required to expand their project plan into an Software Requirements/Requirements Specifications (SRS), which in this context is a mixture of a Game Design Document and a SRS document.

Games Requirements Doc.	Mark
Document convention/intended audience	0.25
High Concept	0.5
Definitions/References/Overview	0.25
Core Experience	0.5
Target Audience & other User Classes	0.5
Target Platform	0.25
User Documentation/Training & Help	0.25
Learning Outcomes: description and measurement	0.5
Other non-gameplay Functional Requirements	0.5
Core Gameplay Features	1
NonFR (security, performance, access, etc)	1
Design/Implementation Req (platform, language, etc)	0.25
Usability Requirements (ease of use, training, etc)	0.25
Client/Partner Feedback – was received by the client reasonably before the due date & feedback from the client was given & included	1
Follows Format Specified/Well-presented	0.5
TOTAL REQUIREMENTS	7.5

The following points describe sections that would be considered "Game specific".

- **High concept** - A concise statement of what kind of a game it is, focusing on what the player does and what they experience.

- **Core experience** - A detailed explanation of what your key experiential goals are. Is your game challenging and exciting? Relaxed and creative?
    - While mentioning the types of fun taught in past units is sensible here, your game might be looking at an additional/unique *core experience* (e.g. Relaxation)

- **Learning outcomes** - What are you trying to teach your player, what are you trying to get them to do? How does it change the player's understanding or ability? Does it motivate the player to do some task?

- **Target audience** - Usually expressed as a persona, with specific preferences, needs and requirements. E.g. “Malcolm plays puzzle games on his iPhone while travelling on the train to work”. This gives you some constraints on who your game appeals to, where/when they are going to play it, for how long, etc.

- **Target platform/interface** - The device your game is going to be played on, including considerations such as screen-size, controllers (mouse, keyboard, touchscreen, gamepad), graphical capabilities, etc. Also include these are appropriate for the kind of game you want to make - these can change as you develop, but you should always be working with a target platform/interface in mind.

- **Core Features** – Essentially a product overview, this should be written as an outward-facing overview of the games main features, similar to an ‘App Store’ or Product listing (on Steam, itch etc) that should clearly highlight the main features of your game – note that features are more than just the mechanics. Think about this from a marketing perspective/reflecting on your client.

For any sections not covered above/that are software specific (e.g. NonFR), please see the Software section (immediately below). Marks will be heavily deducted if they do not follow software formatting for these sections (e.g. FR01 – The system shall….)

Disclaimer: It should be noted that in terms of project lifecycle, you would have already been prototyping your game, as well as finding art inspiration (sourcing references, creating moodboards, etc), as this is part of the assumed process (See **D3 & D4 Project Documentation – GAMES - Game Design Artefacts**). As a result, if you are using Generative AI to assist with art inspiration, please outline your use of it in ‘**Definitions/References/Overview’**

Please follow the following format:
1…...Introduction
1.1 Document Convention/Intended audience
1.2 Game High Concept and Scope
1.3 Definitions, Acronyms, and Abbreviations
1.4 References
2 .....Overall Description
2.1 Core Experience
2.2 Target Platform
2.3 Target Audience and other User Classes and Characteristics
2.4 User/Training Documentation
3 .....Requirements
3.1 Intended Learning Outcomes
3.2 Core Gameplay Features
3.3 Other non-gameplay Functional Requirements
3.4 Design and Implementation Requirements/Constraints
3.5 Usability Requirements
3.6 Other Non-functional Requirements

Details for SOFTWARE projects

(6-20 pages)

If your project concerns software (e.g. desktop or mobile application, website, game) you will need to create a Requirements Specification. If the product already exists (from a previous team or off the shelf product) you should reverse engineer your own requirements document to understand the product specifications.

Software SRS/Requirements Doc.	Mark
Document convention/intended audience	0.25
Purpose/Project Scope & Context Diagram/	0.5
Definitions/References/Overview	0.25
Product Perspective/Features/	0.5
User Classes and Characteristics/	0.5
Operating Environment/	0.25
User Documentation & Help	0.25
Functional Requirements (FR) "The system shall …"	0.5
FR Uniquely identified	0.25
FR Organized/structured/grouped	0.25
FR Covers functionality	0.5
FR Unambiguous/testable/consistent	0.5
NonFR (security, performance, access, etc)	1
Design/Implementation Req (platform, language, etc)	0.25
Usability Requirements (ease of use, training, etc)	0.25
Client/Partner Feedback – was received by the client reasonably before the due date & feedback from the client was given & included	1
Follows Format Specified/Well-presented	0.5
TOTAL REQUIREMENTS	7.5

System/Software Requirements Specification (SRS)/Requirements Document

The SRS should use the IEEE standard as a guide, and should follow the structure outlined on the following page

1…...Introduction
1.1 Overview/Document Convention/Intended audience
1.2 Purpose (of software)
1.3 Scope – including a context diagram (Level 0 Data Flow Diagram)
1.4 Definitions, Acronyms, and Abbreviations
1.5 References
2 .....Overall Description
2.1 Product Perspective
2.2 Product Functions
2.3 User Classes and Characteristics
2.4 Operating Environment
2.5 User Documentation
3 .....Requirements
3.1 Functional Requirements
3.2 Design and Implementation Requirements/Constraints
3.3 Usability Requirements
3.4 Other Non-functional Requirements (Ensure that you specifically discuss security)

Follow the above headings but refer to the IEEE SRS Standard for guidance on what goes into each section. Please do not pay for the IEEE standard which you can access for free if you are logged on to Macquarie University Networks. You must be signed into the MQ campus network for it to recognise the institution. (this does not mean you can login to IEEE with your MQ OneID).

To access off campus use this link:

https://ieeexplore-ieee-org.simsrad.net.ocs.mq.edu.au/document/8559686

The unit website includes some SRS examples that use a range of headings from the IEEE standard. If your client has a format they would like you to use, please use that one and check that it covers the above sections.

Marking scheme for scoping / requirements sections

Marking Scheme for Scoping Document (Data Science and Cybersecurity Only)

For Data Science projects	For Cybersecurity projects	
Document convention/intended audience	Document convention/intended audience	0.25
Purpose/Project Scope & Context Diagram/	Purpose/Project Scope	0.25
Data Understanding – describe the data	Attacks/Vulnerability Understanding	0.5
Sources	Adversaries	0.5
Capture	Attacks/Vulnerability Detection	0.5
Storage/Environment	Monitoring of Assumptions	0.25
Data quality	Data quality	0.5
Data Preparation - cleaning needed	Data Preparation	0.5
Integration needed	Data (packages, logs, …) Collection	0.25
Formats needed	Security Feature Extraction	0.5
Data Exploration – Modelling Modelling technique selection	Attacks/Vulnerability Analysis	0.5
Model design constraints	Defence countermeasures	0.25
Model assessments needed	Architecture, Algorithms, Models,	0.25
Evaluation Requirements & Process	Security Testing (e.g., penetration testing)	0.5
Deployment – monitoring, reporting, review, training/support)	Deployment – monitoring, reporting, review, training/support, security polices)	0.5
Client/Partner Feedback	Client/Partner Feedback	1
Follows Format Specified/Well-presented	Follows Format Specified/Well-presented	0.5
TOTAL REQUIREMENTS		7.5

Marking Scheme for Requirements Document (Software / Web and Games Streams Only)

For Application/Website	For Games	
Document convention/intended audience	Document convention/intended audience	0.25
Purpose/Project Scope & Context Diagram/	High Concept	0.5
Definitions/References/Overview	Definitions/References/Overview	0.25
Product Perspective/Features/	Core Experience	0.5
User Classes and Characteristics/	Target Audience & other User Classes	0.5
Operating Environment/	Target Platform	0.25
User Documentation & Help	User Documentation/Training & Help	0.25
Functional Requirements (FR)“The system shall …”	Learning Outcomes: description and measurement	0.5
FR Uniquely identified	Other non-gameplay Functional Requirements	0.25
FR Organized/structured/grouped		0.25
FR Covers functionality	Core Gameplay Features	0.5
FR Unambiguous/testable/consistent		0.5
NonFR (security, performance, access, etc)	NonFR (security, performance, access, etc)	1
Design/Implementation Req (platform, language, etc)	Design/Implementation Req (platform, language, etc)	0.25
Usability Requirements (ease of use, training, etc)	Usability Requirements (ease of use, training, etc)	0.25
Client/Partner Feedback	Client/Partner Feedback	1
Follows Format Specified/Well-presented	Follows Format Specified/Well-presented	0.5
TOTAL REQUIREMENTS		7.5

Deliverables 3 & 4

Common sections for all groups:
Deliverables Certificate
Updated Project plan (with revision history + new "Handover Requirements" section)
Updated Requirements/ Scoping Document (with revision history)
Project Documentation
Prototype / Product (MVP)
[D4] Team Retrospective
[D4] stream-specific user document
(Feedback for Deliverable 3, 40 marks for Deliverable 4)

ASSESSMENT 2 - Team Project Rubric

Revised Project Plan	3
Revised Scoping/Requirements	2
Project Documentation	20
Prototype/MVP	15
Professionalism (Not well-presented, spelling mistakes, language not businesslike)	PENALTY
Deliverables certificate not included	PENALTY
Project Presentation [Deliverable 5]	10
TOTAL	50

All Teams: Deliverable 3 & 4 submission instructions

Your Deliverable submission should include **ONE** (1) *.pdf* file, containing multiple documents. For a breakdown of what is required by each section, see below. For your submission, include your team number in the file name, and structure your *.pdf* in this order:

1. **Revised Project Plan** – the complete plan, with your changes, and revision history, and inclusion of any corrections/updates to *Schedule* or *Assumptions*.

2. **Revised Requirements/Scoping document -** include the complete document with your changes and revision history.

3. **Project Documentation** – unique per project stream. See the below for more info.

4. **MVP/Prototype Document** – a report showing project progress (with images).

5. **Team Retrospective** [Deliverable 4 only]

6. **Stream Specific document** [Deliverable 4 only] – Again, see the below sections for further information. Depending on your stream, this document will be a:
   - **Software** - User Manual
   - **Games** - Help/Training manual
   - **Data Science** - Scripts/Model Execution document
   - **Cybersecurity** - Recommendations/Monitor Report

Each of these documents should have their own Table of Contents, however if it suits your formatting better you may have just one TOC at the start. Please use hyperlinks to jump to each part of the document. Each document should have its own revision history. You could create these *.pdf* documents separately and then do a merge if you have the right software. Include Deliverables Certificate as the first page.

Warning- Submission of documentation should be done by the due date via iLearn. Documentation must be consistent and relevant for the project and prototype demonstrated. The client typically is interested in the product, not the documentation. A poor prototype or redundant/inconsistent documentation will bring down the documentation mark.

D3 & D4 Common sections for all groups

Updated Project Plan

The following is required:

1. **Revision Table**: Please add a revision table to the start of this document, just after the TOC, to indicate the revision number, date, person and change to the previous version (be specific about what changed / why). This should be updated again in Deliverable 4.

2. Add the new section **Handover requirements [Include the heading in the project plan]**. This section should be added after your *Schedule* section, and should explain in detail what the client needs to be given.

   This section also needs to clarify the intended nature and content of the stream specific mandatory part of D4 i.e. User Manual OR Help/Training OR Scripts/Model Execution; OR Recommendations/Monitoring Plan.

   Note that the handover will be different according to what the project is. It's about making sure the client gets what they were expecting and need before the team disappear. So any data, software, algorithms, scripts, test cases, testing tools/environments, outputs and documentation would be handed over together with training/manuals that lets them go forward with it - including reports or test cases showing current output and/or what is fully functional or not.

   For all projects, the point is to discuss with the client what they need at the end of the project. Make sure they have everything they need and know how to use what you have given them. There is no set format or requirement for the marker.

   For games and software projects, think about:
   - Who will do play testing and/or acceptance testing, asking questions such as when will testing start,
   - When and what devices/sites does the final product need to be deployed on,
   - Who will maintain the system,
   - How will the system be maintained,
   - What are the user manual / training requirements & parameters
   - What database/file setup is required, what documentation is required.

   For cyber and data science projects, think about how the client will use what you have done. Do they need a list of recommendations and actionable items, scripts for rerunning machine learning algorithms/updating visualisations or vulnerability tests, a monitoring report to ensure processes are followed/tests are run, etc.

3. **A revised and updated version of your project *schedule*,** that takes into account any schedule revisions, covering the balance of the project, that have arisen since you submitted the previous schedule. Anything else that has changed such as team structure, resource allocation, risk evaluation, change management processes, communication

strategy, etc should also updated in the plan and quality manual and included in the revision history. Resubmit the entire updated document.

Updated Requirements / Scoping Document

While no new sections are required, please add a revision history table at the start of your document to clarify all changes made from Deliverable 2, and please clearly identify improvements and changes. Each submission should involve a team review of the document, review of feedback from previous Deliverables, and the inclusion of any project changes or sponsor feedback. Note that this revision table should be updated again in Deliverable 4.

Project Documentation (Summary – See Individual Sections)

Please read the appropriate/stream specific sections further below for specific information.

For Deliverable 3:

- The first step is determining what project documentation is needed – **you may find that you need to change project streams for Deliverable 3 & 4**. Every project is different. Please use the knowledge you have gained in other units and through industry experience together with stream specific information and rubrics to determine what documentation you need. Speak with your industry partner to ask them what documentation they require/suggest.

- **You may find that a rubric section/heading for your relevant stream is not relevant to your project** (e.g. A software project that is done entirely within a terminal will likely not need Screen Navigation diagrams). **If you deem a section as not relevant to your project you do not have to complete it** (the marks will be redistributed across the Project Documentation section), but please include the heading and clarify to the marker why it is not relevant.

- **You might find your project could require marking in two or more streams.** If your project includes more than one stream (e.g. a dashboard that presents various data visualisations, a gamified website, software to manage security issues) you may draw on multiple stream specific sections and rubrics to determine what documentation is relevant to your project. Please clarify to the marker what streams you have drawn on. You do not need to justify all the sections you chose not to use, but you can particularly if you would like marker feedback on your decision/reason.

For Deliverable 4, you will need to update your Project Documentation. You will receive feedback on your submission for Deliverable 3, and will be required to update Deliverable 3 according to feedback, project progress and changes for assessment in Deliverable 4.

Prototype / MVP Document

Before the due date/time you need to arrange a time with the client to demonstrate your system/present your current progress and receive feedback. The team must organise that meeting**,** and when the demonstration/meeting is arranged please let the convenor know of the arrangement via the meetings thread on iLearn (as well as a follow-up in the meetings thread if necessary). **All team members are expected to attend** - A valid reason for non-attendance by a team member must be arranged before the meeting and included as an in aforementioned weekly report (or added to the forum thread).

Under the heading *Prototype*, include at least three pages in your Del3 and Del4 documentation to present and discuss your prototype:

Also include:

- A subheading "Sponsor meeting, feedback and response to feedback" in the "Prototype" section.
- Details of the sponsor meeting and feedback - All sponsor feedback on any aspects of your project should be included in this section.
    - Also clarifications how the team have or will respond to that feedback.
- Evidence - Code snippets, screen shots, how the product looks and works, scripts, database schemas and other evidence of your prototype progress should be included – this document should clearly and concisely show evidence that convinces the marker what you have done so far, and match what the client has been shown.
    - All evidence should be appropriate annotated
    - For MVPs which are reports, also include samples of your report.

If you have not been able to arrange a meeting before submission due to sponsor unavailability state that under this heading and when the meeting is scheduled. After the meeting, include the feedback and response in your next weekly report submitted on iLearn.

I will also be contacting the sponsor - the client will be asked to confirm the meeting and their feedback via a survey sent to them. More information about this can be found in the section below.

Mark Breakdown
The following table outlines the marks given for what is given in the Deliverable itself (up to 10).

Prototype / MVP Document	**Mark**
No evidence or mention of a prototype found in Deliverable	0
Extremely brief evidence, may be scattered/not in a literal section	3
Evidence provided is quite brief	6
Prototype shown with screens and feedback (and possibly responses to feedback).	8
Shown in great detail with screens, feedback, and responses to feedback	10

Additional marks will depend on the feedback outlined in the Deliverable, and the client's feedback to the convenor – note that if the sponsor is not happy, marks may be deducted.

Team Retrospective [D4 ONLY]

(4-8 pages)
As a team, you should review all aspects of your project (information, outputs, documents, weekly reports, meeting minutes, etc) and reflect on what you did, why you did it, what worked and what should have been done differently - structure the reflection so that it is easy for the marker to identify your responses to these.

You may include information / diagrams / snippets from other documents (or otherwise) to make your discussion more informative. There is no set structure, but one approach could be to reflect upon each of the phases you went through: feasibility; project planning and scoping/requirements; design, implementation and testing; handover. Also, consider the unit learning outcomes and how the team have achieved these.

D3 & D4 Project Documentation - DATA SCIENCE

Students choosing the Data Science stream can leverage what was built up in data science units and what has been done in their project. The aim here is to communicate different aspects of the project implementation including the data, the goals, the options, the decisions, the choice of models and starting configurations, and the resources available. For further context, someone else coming in late to the project should be able to use this deliverable to understand what the team is doing and where they are up to with their deliverables (including the rationale behind any design decisions) – in terms of how informative this deliverable should be.

Section	Default Weighting
Project Documentation	
Feature Engineering (selection, construction)	3
Solution Architecture	2
Algorithm/Models/Methods	2
Detailed Data descriptions	2
Test Specification	
Model Evaluation	3
Performance Evaluation results	2
Deliverable 4 Additions	
Scripts/Model Execution	3
Team Retrospective	3

For DATA SCIENCE projects, you should seek to establish a baseline model for the project – the simplest model you can think of for the problem. This would be a good deliverable as a MVP since it encourages you to have an end-to-end processing pipeline and to establish a baseline that you can then try to beat with their refined model in the subsequent increments.

Your documentation should include the data manipulation process with clear instructions on how to run them to turn the raw data into the input for modelling. One possible approach could be to load / maintain them in a GitHub / Bitbucket / Firebase repository with appropriate permissions shares. However, this is something that should be discussed with your sponsor and the convenor.

You should present the results of the modelling via evaluations in the form of a report. The report should outline the different models used and the results observed. Interpreting the results and discussing implications for the business questions that prompted the study are also a key part of this. In many cases this can be structured like a traditional scientific paper – material, methods, results, discussion.

Feature engineering (deciding what you are looking for and how to go about it):

- From the data, are there any trends / ranges to look for?
    - e.g. if you are analysing time-based movement data, what accelerometer and gyroscope ranges / trends do you want to look for to identify someone that has fallen / collapsed?
    - e.g. if you are looking at financial data, what does an "upward sale trend" look like?
    - e.g. if you are looking at population data, or education rates, what "characteristics" would you want to look at and why?
- Describe what data characteristics are being looked for, and how your data pipeline is being processed to generate these features.
- Give each different "feature" or "characteristic" a name and then attribute some form of data ranges / statistical definition.

Solution Architecture (choice of macro architecture / pipeline):

- Provide an overall description of each section of the pipeline including the data in and out of each "stage". The data details can be properly described in the "Detailed data descriptions" section.
- This would most likely be a more detailed version of the overall pipeline presented in the team's scoping document as some of the stages would now be implemented / finalised.
- Include any resources available / resource processing constraints to each of the sections in the pipeline (e.g. processing / timing limits)

Algorithms / models methods (detail what is in each part of the solution architecture, including models used and initial conditions / config settings):

- selection of model... there are many different approaches: predictive, supervised / unsupervised, classifiers, ... which are going to be used? why? and why chose those over other approaches?
- Are there any settings needed (eg. in a KNN model, what is the number of the nearest nodes being used in the application of the classifier?)

Detailed Data descriptions:

- Data being used, data being generated, data being stored, as well as any summaries and/or reports.

Model evaluation:

- how will any data / outputs be compared / tested / evaluated for correctness and accuracy?
- if you are choosing between models, how will the models be compared / contrasted to see which one has a better performance (e.g. if you are comparing different classifier models, on what basis are you comparing them? Detail each comparison.

Performance evaluation results:

- What are the results of any tests run so far
- What are the future planned tests for future iterations? (e.g. if this is Deliverable 3, what is planned for Deliverable 4? IF this is Deliverable 4, what is planned before the handover?)

Deliverable 4 (Only) stream-specific document: for Data Science Teams (5-20 pages)

Scripts / Model Execution

The *Scripts and Model Execution Documentation* should not exceed **twenty (20) A4 sides.** This is essentially a user or training manual for the one or more users of your MVP. It might include model parameters, how to set up the models, data sources/format, file locations, etc. The intended user/s may differ according to the project and there could be more than one type of user with different documentation needs.

The content of this document is something that you MUST discuss before **Del 3** when you talk to your client about what they expect at handover at the end of the project See D3 & D4 Updated Project Plan – point 2), ask them what their requirements are regarding support/maintenance. Find out what they need regarding training, help, installation, configuration, etc of your product/output.

Like all technical writing, the document should have a clear Table of Contents (TOC) and well-organised content. In addition to documenting your models and how to run them, this document could involve sections for different types of users (e.g. Admin and end users), installation guide, configuration settings, screen shots with example data, APIs, training, steps describing usage troubleshooting/where to get help, etc. Check you have page numbers, or your TOC will have a limited value.

D3 & D4 Project Documentation - CYBERSECURITY

Discuss with your client what they want to receive from you – that will be your MVP. It could be a *Penetration Testing Document* as in the example provided on iLearn. It could be a list of recommendations and vulnerabilities discovered. It could be an overview of attacks or defences tried (if you are doing red team / blue team approaches).

Section	Default Weighting
Project Documentation	
User stories describing security/privacy attacks	3
Detection of intrusion /vulnerabilities	2
Defending Solutions Architecture	1
Non-technical concerns/approaches	1
Algorithms/Security Rules	1
Data Description/Feature Engineering	1
Test Specification	
Security Test Plan	3
Testing Reports	2
Deliverable 4 Additions	
Recommendations/Monitoring Plan	3
Team Retrospective	3

In addition to the MVP you provide to your client, you will need to provide project documentation as part of D3 and D4. Refer to the rubric for content and sections to be included in the analysis, design and testing documentation. Below is guidance describing the possible subsections of your documentation.

User stories describing the security / privacy attacks:

- What are the points of interest in the system and what attacks were considered / attempted?
- Be sure to include the sections / areas that these were applied.

Detection of intrusion / vulnerabilities:

- What did you find and how were you able to determine that the intrusion was successful / a vulnerability was discovered?

Defending solutions architecture:

- What was the architecture of the system under focus?

- Also, describe the mechanisms / process flow for your own investigations / attacks / defences (depending on the mode of your project). This could be represented as some form of "pen-testing architecture".

Non-technical concerns/approaches:

- Describe other cybersecurity aspects such as governance, human-factors (e.g. social engineering), processes, staff training, etc, that will impact on the security of IT infrastructure for the organisation.

Algorithms / security rules:

- Describe the pen-testing algorithms / approaches used (or the defending equivalent if you are focusing on that approach). For example,
  1. Provide a high-level overview of each attack or "shadow" model – describe its purpose, inputs/outputs, control flow and how it fits into your overall offensive or defensive strategy.
  2. Outline your rule-sets or detection heuristics (e.g. what patterns you're matching, thresholds you've chosen, any signature-based or anomaly-based checks).
  3. Use pseudocode, flow-charts or UML/activity diagrams to illustrate the key steps or decision points of your algorithms.

Data description / feature engineering:

- What data have been collected / used / provided?

Security test plan:

- This is a detailed overview of the entire testing intent for the project including: Timelines, approaches, resources required, times, people involved, approvals, reporting needs, overview and list of test cases. Refer to sample documentation in iLearn for general test plans that could be modified for security projects.

Testing reports:

- Detail the specifics for each test case what / when / where / how …
- What results have been found so far and what remains to be tested / followed up? This can be a comparison to your initial planned scope.
- What tests are yet to be completed / attempted?
- Are there any new tests / approaches being considered after having discovered more about the system under focus?

Be careful not to focus only on the technology side of the matter, rather than taking a holistic view. It may work well if the context and client can support this approach. However, in cybersecurity projects where the client is needing cybersecurity support but not wanting the team to conduct penetration testing or other technical changes to their current system, the team may need to think more broadly about cybersecurity and how to assist their client to make their systems/business more secure, now or in the future. However, while cybersecurity should still be the focus, there should also be a holistic view in some aspects. What are the major security concerns in this service, for example the type of users (admins without strong IT background, general public, school kids).

Note that it is quite common that sometimes clients don't even know what they want. Clients may have a security concern in mind, so they come with a security project, however, the reality is that they need a holistic IT solution instead. For example, if the organisation was having issues with their current web hosting service and are looking for a new solution provider, there wouldn't be much value in conducting pen-testing. Instead, the team should first focus on requirements engineering and help the client to reconsider the focus of their proposed project.

Additional notes

- If you are not doing a pen-testing focused project, ensure in the testing section to provide explanation of any changes and discussions on alternatives, rather than pen-testing specifics. Think about What can be tested - for example, the team can focus on the development of suitable metrics for measuring effectiveness of policies.
- The marker will be looking for evidence of some structured and professional approach resulting in you making these recommendations - i.e. not simply "this is a list of good ideas we brainstormed", but a list of recommendations that are logically concluded from evidence that is presented in a professional/industry standard format. Provided below are some examples of ways to structure your evidence, keep in mind your client may prefer you to choose something specific/something different (you should note this if that is the case):
    - Usage of common cybersecurity frameworks and standards, like the NIST Cybersecurity framework or the NIST Privacy Framework
        - If you choose this option, you may prefer to use a resource like CSF Tools
    - Methodologies, like Threat modelling or Attack trees
    - Threat catalogues, like STRIDE or the MITRE ATT&CK framework

Deliverable 4 (Only) stream-specific document: for Cybersecurity Teams (5-20 pages)

Recommendations / Monitoring Plan: Provide an early draft of recommendations / results reported so far based on findings. This should not exceed **twenty (20) A4 sides.** The intended recipients will differ according to the project and who is the target recipient of your MVP (e.g. the sponsor's organisation or one of their clients). The document should have a clear Table of Contents (TOC) and well-organised content. Check you have page numbers, or your TOC will have limited value.

The content of this document is something that you should discuss when you talk to your client about what they expect at handover at the end of the project See D3 & D4 Updated Project Plan – point 2), ask them what documentation they require to support the MVP such as executing, monitoring, training, help, installation, configuration, etc of your product/output.

D3 & D4 Project Documentation - GAMES

Section	Default Weight
Project Documentation	
GDD - MDA Framework	2
Art book	1
Storyboards	1
UI mock-ups	1
Additional Artefacts	1
Software/System Design Document - Core	1
Additional Scaffolding	1
Data Definitions	1
Test Specification	
Playtest plan & Functional Test Plan	3
Playtesting Results & Test case Specifications	2
Deliverable 4 Additions	
Help/Training	3
Team Retrospective	3

Your Project Documentation is split into three major sections: A Game Design Document, Game Artefacts, and Game Scaffolding. **Check iLearn for templates, such as the Playtesting Report template and Test Case Specifications.**

Game Design Document – MDA Framework

Should contain four sections, referring to the MDA framework:

1. **Mechanics (Rules)** – A clear list of the mechanics of the game (with clear, brief descriptions), highlighting the rules that implement the game systems.
2. **Dynamics (Play) –** The intended behaviour that the mechanics are intended to produce during play (for example, intended strategies). You may wish to write a long-form response, or simply create a list of patterns of play
3. **Aesthetics/Experiences (Fun) –** An outline of your experiential goals (outlined in your SRS), with an explanation of how your mechanics/dynamics aim to achieve them. This section should explain what explain what your game fun
4. **Other** – If you have core components to your game that don't distinctly fit into any one part of the framework (or they fit in all parts of the framework), mention them here instead. Examples may include progression systems, story/narrative, technology (e.g. VR Design implications) etc**.** Please ensure that these items are not more appropriate in the System Design Document before including them here.

Game Design Artefacts

Gameplay Storyboards

A 'flip-book' detailing how you expect users to use game mechanics frame-by-frame. Illustrations should aim to focus on how players achieve experience goals. Refer to your notes from COMP2150 for how to achieve this.

Artbook

Your artbook should be split into two parts:

- **Inspiration** - Concept art and/or images from **other** games or media that represent the art style you have in mind for the game. **It is acceptable to include Generative AI for this** if you used it as part of your design process.
- **Created Assets** - Any depictions of art assets you have made for the game, as well as iterations of models, levels and anything else sensible for this section (e.g. Concept Sketches)

UI Mockups

These UI mockups should show the important information elements and how they are communicated to the player. Keep in mind these are not quite storyboards, but more akin to snapshots that are designed/iterated upon, focusing on the contextual information that the player can see during gameplay.

This should already be an artefact of your design process, as you should be considering the player information displayed during the design process. Additionally, it is recommended to include screen-flow diagrams if appropriate (e.g. for important sub-menus)

Additional Artefacts

As further evidence of your development lifecycle (using Agile), you may wish to include some of the following (Note that these are suggestions; it is fine if you have no additional artefacts):

- Design mind maps
- Molecule Diagrams
- Paper Prototypes (evidence of playable versions of the core gameplay loop, implemented using cards, tokens, dice etc)
- Additional/Software Prototypes (lo-fi implementations of mechanics made during development to "find the fun")
- (… or anything else you deem appropriate)

Game Scaffolding a.k.a. Software/System Design Document

This should contain three sections:

1. **Core Document** - This will include the basic architecture of the system and the high-level strategy decisions. Use the following headings and explain if and why it is not relevant for your project, if that is the case – it is very possible you will not need some of these. You need to include a description and justification of the:
    a) System architecture – including package diagrams with description that is consistent with the system architecture
    b) storage/persistent data strategy – e.g. relational database versus flat/text files
    c) any concurrent processes or data and how they will be handled if any exist
    d) user interface/user experience strategy – e.g. tracker, spoken dialogue, phone input, form/menu-based, GUI, etc.

e) design decision choices and trade-offs – e.g. Changes in fidelity to hit performance requirements, restrictions based on technical constraints

2. **Additional Scaffolding** – Design of out-of-game features. For example:
   - Leaderboards
   - Permissions/Authentication
   - Features for Developers (e.g. Dev Tools), or features for UGC (User-Generated Content)

   To document these software features you may use activity diagrams for intended flows/algorithms, use case diagrams (for different out-of-game features), etc. You can refer to the software development section to see more about documenting software.

3. **Data Definitions** – Create a table showing what data is needed to be stored in files/database, or onboard the device. Include data field name, type and example.
   - For relational databases, for each table/file show the name of the field, the primary key (if applicable), the field type and an example of data in this field. E-R diagrams are recommended.
   - This might also include save data, required cache data, logs, etc.

Game Testing deliverables [Test Specification]

The game testing deliverables adheres to both Software (Test Plan/Functional Test cases) and Game (Playtesting) requirements.

Playtesting

Aimed at testing game features and experiential goals, this is required to assess your implementation of gameplay features outlined in your SRS, in terms of them matching your design goals.

Disclaimer: While ideally playtesting should be done at all stages of development (hence the need for rapid prototyping), **it is expected that by the submission of Deliverable 4 that you have completed at least ONE formal playtest with at least THREE sessions with external members/participants (minimum 1 per session)**. Note that this is the minimum, and that two sessions for a playtest will likely not even give you enough data to answer your design question with clear evidence – if this is not possible, please justify your reason as to why in the Playtesting Results section (You are still expected to write a Playtest)

You can include multiple Playtest plans, but please only include results for one of them. Your playtest should be structured as follows (with additional sections included, if relevant):

1. **Design Question(s) / Purpose** – A brief breakdown of what you're expecting to find/answer from this playtest. Some preamble/additional context being added (i.e. an Introduction) might also be relevant
2. **Prototype Specifications** – A brief description of the build being used, as well as if certain parameters are set/being used (e.g. An outline of the differences between versions if A/B testing is being performed)
3. **Sample** – An outline/brief description of the sample (i.e. the people you are having playtest your game)

4. **Method** – An outline of the instruments and methodology of your playtest. Types of data being gathered (e.g. Quantitative, Qualitative), and techniques used to gather data (e.g. Interview, Survey, Observation) should be specifically mentioned.
5. **Results** – A summary of your key findings and outcomes. Note that this does not include ALL of your results, which should be present in your appendix
6. **Conclusion** – Analyse and attempt to explain the results and the significance of the findings. **This should also include an action plan** for applying findings to your overall game design, or implementing other feedback (if appropriate).
7. **Playtesting Appendix** - Raw results, testing artefacts/templates, etc)

Software Testing

Aimed at testing game functional and implementation requirements – the exact same requirements outlined in your SRS. This section is the same as the Software stream (although it is assumed you will have far less things to test):

1. **Functional Test plan** – aimed at testing ***Functional requirements*** including testing strategy (white/black box) and types (unit, system, penetration, regression, etc), testing tools and resources assigned. This test plan should also include a detailed test schedule (with calendar dates, not university weeks), featuring testing milestones and test deliverables, covering scheduling and resourcing of all testing processes.

2. **Test case specifications** – For each test case specification, if a test has been completed include your latest results (i.e. actual result/output, pass/fail, etc); we would like the state of these to reflect that testing is ongoing. This test-case specification should include the following:
    a) Identifier(s)
    b) Test description.
    c) Input specifications (e.g. actual input to be tested or link to be tested - include range of test values i.e. possible valid and invalid input)
    d) Output specifications (e.g. expected outputs)

Deliverable 4 (Only) stream-specific document: for Games Teams (5-20 pages)

Help / Training Documentation

The *Help / Training Documentation* should not exceed **twenty (20) A4 sides.** The intended user/s will differ according to the project. For most games or software projects, the document should enable a moderately computer-literate user, initially completely unfamiliar with the system, to understand and fully utilise its functionality. Note there may be more than one type of user with different documentation needs. Training to use the game could also be provided as part of the game. This could involve special keys and screens with help and/training videos. Screenshots of help/training or the training video could be provided in place of or in addition to creation of a document. Please confirm alternatives to documents with the unit convenor well before any deadline. It is likely you need a combination of both documented (for installation, management and maintenance) and in-game support.

The document should have a clear Table of Contents (TOC) and well-organised content. Content will depend on the project but could involve sections for different types of users (e.g. Admin and end users), installation guide, configuration settings, screen shots with example

data, APIs, training, steps describing usage troubleshooting/where to get help, etc. Check you have page numbers, or your TOC will have limited value.

As part of the discussion that you should have already had with your client about handover to them at the end of the project (See D3 & D4 Updated Project Plan – point 2), ask them what their requirements are regarding support/maintenance. Find out what they need regarding training, help, installation, configuration, etc of your product/output.

D3 & D4 Project Documentation - SOFTWARE

Section	Default Weight
Project Documentation	
Use case diagram	1
Use Case descriptions and User Stories (Well-structured, sensible, complete)	2
System Design Document	2
Design – includes UML models, structure charts, Report Layouts	2
User Interface Layouts, Screen Navigation	1
Data Definitions/Schemas/ER	1
Test Specification	
Test Plan	3
Test-case specifications	2
Deliverable 4 Additions	
User Manual	3
Team Retrospective	3

Your Project Documentation is split into two parts – Analysis and Design

Analysis Documentation

Use Case Diagram

This is a graphic model showing the actors, the use cases and the relationships between them. One page should be sufficient. As a rule of thumb – if the description of a use case is very short (e.g. one or two steps only) or very similar to another use case, then consider combining the use cases and describing the alternate courses of action within that use case. Structure the use cases into logical groupings. Remember it's from the users' point of view – not the developers.

See the Resource section on iLearn guides on how to draw Use Case diagrams and common misunderstandings and errors.

Use Case Descriptions (one for each use case in the use case diagram)

Please use the template on iLearn

A use case is a chunk of functionality. You must have a use case description for each use case. This will elaborate all the ways (uses and the steps to achieve them) in which people will achieve them. If you have many use cases, possibly some of your use cases are actually steps in a use case, not a chunk of functionality. e.g. if they just have one step (like "save", "submit", "authenticate"), they are probably a step in another use case and not their own use case.

When you start to write the use case steps, it will become more obvious whether you have identified separate chunks. Also, if you are repeating steps, then this is a candidate for reuse and should be an "includes" use case. Check you are not joining use cases together in sequences. Only includes, extends and generalisation are valid lines between use cases. E.g. “login” should not be joined to other use cases since you could login and then do nothing, or choose from a range of functions/use cases.

If you have 10 chunks of functionality then you need 10 use case descriptions. How will you know what to implement, if you don’t know what triggers each use case, which user initiates, what the steps are, etc?

Please use the template on iLearn. Sub-use cases can be combined into one use case description. For example, if you have a use case Maintain Client with sub-use cases Adding Client, Deleting Client, Viewing Client or Modifying Client, you can just write one use case description for Maintain Client and describe the differences by branches in the use case steps.

User Stories

In line with agile practices, write user stories for your product. See the following link to get some tips: http://en.wikipedia.org/wiki/User_story. You can list these in a separate section or add the appropriate user story in your use case description.

Design Documentation

Design Documentation can include the following..

1. System Design Document
2. User interface layouts / Report layouts (if needed)
3. Window Navigation Diagram (be sure to show backtracks / alternate navigation paths)
4. Data definitions
5. **For every use case description** create:
   Activity Diagrams
   OR
   Sequence Diagrams (if OO programming language being used)

If classes/objects are being used:

6. Class Diagram
7. State Diagrams for objects with interesting behaviour

System Design Document

This will include the basic architecture of the system and the high level strategy decisions. Use the following headings and explain if and why it is not relevant for your project, if that is the case. You need to include a description of the:

a) system architecture – including package diagram with description that is consistent with the system architecture
b) storage/persistent data strategy
c) any concurrent processes or data and how they will be handled if any exist
d) user interface/user experience strategy - e.g. tracker, spoken dialogue, phone input, form/menu-based, GUI, etc
e) design decision choices and trade-offs – e.g. changes in design to hit performance requirements

User Interface Layouts
Show the actual screens or use a drawing package, system builder like e.g. Jbuilder, interface designer tool.

Report Layouts
If you are producing any reports (e.g. weekly sales report, daily summary, transaction listing), you need to design them. You need to consider things like report title, column/page headers, report totals/subtotals, fields, how many per page, etc). Different types of reports have different real estate (e.g. paper size or screen size). For example, if you needed to print a sales docket you need to plan what goes at top and bottom (e.g. name of company, ABN, store location, sales person's name, payment method, etc) how many spaces you have across and down and how to layout the docket so it will fit on the paper or screen (e.g. computer/mobile) to be used.

Again**, it is possible that your software will not require 'Report layouts'**. If that is the case, please explain why it is not necessary for your project.

Window Navigation Diagram
Show how the different screens will link together (screen flow) and what triggers a transition from one screen to another. Activity diagrams are recommended.

Data Definitions
Create a table showing what data is needed to be stored in files/database. Include data field name, type and example. For relational databases, for each table/file show the name of the field, the primary key (if applicable), the field type and an example of data in this field. E-R diagrams are recommended.

Class Diagram
If you are using object-oriented programming please include a class diagram. Make sure any classes or methods on your sequence diagrams have been included on the class diagram. Method signatures should be given. The diagram must include all of the following:

- classes
- attributes
- associations
- inheritance and/or aggregation (if applicable)
- traversals
- multiplicities

Sequence Diagram
If you **are** using object-oriented programming please include one sequence diagram for each use case description. It should be possible to read the description and follow what is happening on the diagram. The objects and messages must be valid and shown on the Class Diagram.

Activity Diagrams
If you **are NOT** using object-oriented programming please include one activity diagram for each use case description. It should be possible to read the description and follow what is happening on the diagram.

State Diagrams

If you are using object-oriented programming include state diagrams for any objects that have interesting states or complex behaviours. A state diagram shows the life cycle of an object and thus all important objects can each be represented in their own State Diagram. However, you are required to consider the life cycle of each object in your system and to submit diagrams for those that have interesting states or complex behaviour. One way to measure if a state is interesting is to consider whether you need to test that state before performing a particular action or if the state changes after an action is performed. What is interesting will depend on the application. In most cases when an object is updated or printed (updated and printed can be states themselves but are generally not very meaningful) that will not change more interesting states such as paid/unpaid, married/single or for sale/sold.

Testing Documentation [Test Specification]

This is comprised of two sections – Note that templates are provided on iLearn:

1. **Test plan** - including testing strategy (white/black box) and types (unit, system, penetration, regression, etc), testing tools and resources assigned. This test plan should also include a detailed test schedule (with calendar dates, not university weeks), featuring testing milestones and test deliverables, covering scheduling and resourcing of all testing processes.

2. **Test case specifications** – For each test case specification, if a test has been completed include your latest results (i.e. actual result/output, pass/fail, etc); we would like the state of these to reflect that testing is ongoing. This test-case specification should include the following:
    e) Identifier(s)
    f) Test description.
    g) Input specifications (e.g. actual input to be tested or link to be tested - include range of test values i.e. possible valid and invalid input)
    h) Output specifications (e.g. expected outputs)

For templates, check the Resources section on iLearn. Use of these templates is optional and suited to projects delivering a software product. If you prefer, you can use testing documents from previous units or other source; your sponsor may also provide ones for you to use.

Deliverable 4 stream-specific document: for Software Teams (5-20 pages)

User Manual Documentation

The *User Documentation* should not exceed **twenty (20) A4 sides.** The intended user/s will differ according to the project. The document should enable a moderately computer-literate user, initially completely unfamiliar with the system, to understand and fully utilise its functionality. Note there may be more than one type of user with different documentation needs.

The document should have a clear Table of Contents (TOC) and well-organised content. Content will depend on the project but could involve sections for different types of users (e.g. Admin and end users), installation guide, configuration settings, screen shots with example

data, APIs, training, steps describing usage troubleshooting/where to get help, etc. Check you have page numbers, or your TOC will have limited value.

As part of the discussion that you should have already had with your client about handover to them at the end of the project (See D3 & D4 Updated Project Plan – point 2), ask them what their requirements are regarding support/maintenance. Find out what they need regarding training, help, installation, configuration, etc of your product/output.

Deliverable 5 Details

Project Presentation (10 marks)

Each group will be given a 15-minute (12 minutes for presentation and 3 minutes for questions/change over) time slot to present their project to other teams, sponsors, staff and academics in one of the lecture theatres. **You must attend the whole session you are assigned to, not just your project presentation – you must contact the convenor if this is not possible**. Please ensure teach team member puts their name on the bottom of whatever slides they present.

Your presentation should mention and/or include:

- Your sponsor's context
- An outline of problem or opportunity being addressed,
- Your team's initial approach(es) to the problem,
- The value of what you have done to address the problem.
- An assessment of (i.e. a reflection) the project,
- An assessment of (i.e. a reflection) the current MVP
- An assessment (i.e. a reflection) of process
- An outline of future/outstanding work.
- Be sure to finish on a positive note about has been done not about what hasn't been done.

Software presentations should also include a demonstration of the software, and Game presentations should include either a pre-recorded or a live demonstration of their game. Make sure that your demo will run in the lecture room that will be used. Check with the convenor beforehand if in doubt. For demonstrations, it is recommended to show/mention:

- functionality of the product,
- non-functional qualities of the product
- examples of how to the product/system would be used, by whom and for what purpose.

It is advised you use your own device for the presentation, plugging into the console in the room. If your industry partner is online, you will need to arrange to have them join via Zoom or another platform. Note that the visualiser at the lectern can be used to demo applications running on mobile devices.

Your project may involve some confidential information. It is best to discuss this with your client ahead of time, and find a solution around what can and cannot be presented. Speak to unit staff if you are struggling with this.

The received presentation mark will be the average of your individual presentation mark and the group marks awarded for presentation, content and visual aspects (see below); all the metrics overlap to measure how coherent and professional your presentation is. **Please put your name on the bottom of each slide you present so every marker in the room (unit staff, visiting academics, and industry partners) can mark you appropriately.**

A group mark will be given for each of the following criteria:

- **Presentation structure**
  - Does the presentation follow a logical structure
  - How well does the sequence in which the content was delivered, and the flow of concepts assist with giving a concrete understanding of the content/your project

- **Communication of content**
  - How is it delivered (via a demo, video, slides, just talking, etc),
  - How clear your explanation of the project is (ie. through your explanation, was it easy to obtain an understanding of your project)
  - How informative your presentation is
  - Were there any irrelevant points or contradictions

- **Visual aspects of the presentation**
  - How pretty/stylised your presentation was
  - How useful the visual aids were (and/or any demonstrations), for giving further understandings of your project

An individual mark will also be given for **Presentation/speaking skills** - while there are no strict requirements regarding team members needing an equal amount of time, each team member should be given an appropriate amount of time to be appropriately assessed. Dominating the presentation unnecessarily may result in your own individual mark being reduced.

Each item is marked out of 10. The final mark for each team member is individualised - the group mark is averaged across the different criteria, with the received individual mark being a combination of the group mark average, and the individual ‘Presentation skills’ mark.

Deliverable 6 Details

Individual Project Reflection (20 marks)

During the exam period, you will sit an online exam where you will respond to questions in the form of reflective writing. This is an individual task, and more information will be given during the Week 11 class.

SOFTWARE & SUPPORTING MATERIALS

You must discuss with your client what they need to receive from you. Do they need a manual, install wizard (which platforms/OS)? What files/database needs to be set up, how can they run and maintain your system? This discussion should have started before submission of Deliverable 3 and continue.

When you have handed over what has been agreed, the sponsor must check it works on their setup (or you can check that for them). They will provide a mark out of 5 for each individual student to the unit convenor after confirming they have received what has been agreed.

It is not sufficient to hand over a zip file and wish them all the best. You must make sure they can run your system – unless they agreed that they have technical people who will handle this. So you need to make sure your system is fully tested, well documented and bug free before handover. You should arrange a date for handover and allow your sponsor up to week to confirm they can follow your instructions and use the system. Check iLearn for the handover dates. Leaving handover to the last minute may mean you receive an incomplete for the current semester as there may not be enough time for the sponsor to provide a mark and for it to be processed in your end of semester results.

Additionally for Deliverable 7, on iLearn upload a zip file that includes the latest version of all documents created over the semester (i.e. D1 & D4 – since D2 and D3 are already updated in D4). This is primarily an archive for the unit. You should update all documents that you will be handing over to your sponsor based on any feedback since the last deliverable. What is submitted will not be assessed, so teams may submit the latest previously submitted version but preferably the documents updated for the sponsor, i.e. the latest version, should be uploaded.

If your code or other files are too large or the structure is too complex to zip and upload, then you do not need to include them in this upload. For all projects, but particularly if you can't upload to iLearn, please ensure the client has been able to access the files and use them since there will be no other record of them that they can access in the future.